QuasarCert Soundness:

Canonical Formal Specification, Quasar 3.0

Zach Kelling
Lux Industries, Inc.
research@lux.network

Spec freeze: 2025-12-15
Activation: 2025-12-25

Abstract

This note is the canonical formal specification of QuasarCert soundness for the Quasar 3.0 chain activation on 2025-12-25. QuasarCert is a three-lane finality certificate $\mathcal{C} = (\sigma_{\mathrm{BLS}}, \sigma_{\mathrm{RT}}, \pi_{\mathrm{ZK}})$ with each lane bound to a common *certificate subject* that fixes chain id, epoch, round, mode and the per-chain root commitments (P, Q, Z, A, B, M, F). We prove four results with theorem numbers aligned to LP-020: (7.5) classical soundness, requiring an adversary to forge all three lanes against three independent hardness assumptions; (7.6) parallel-lane liveness, allowing the three lanes to be aggregated independently and in parallel; (7.7) post-quantum safety, the standalone PQ predicate $\mathsf{VerifyPQ} = \mathsf{RT.Verify} \wedge \mathsf{ZK.Verify}$; (7.8) subject-binding replay protection, ruling out cross-epoch reuse of any valid cert. We also address the rejection of threshold ML-DSA in 3.0 (App. A), the GPU verifier in LP-132 (§7), and the concrete post-quantum parameter analysis (App. E).

This proof establishes soundness of the QuasarCert wire format defined in LP-020, with constituent layers specified in LP-070 (ML-DSA), LP-073 (Pulsar; Corona-derived lattice threshold kernel with key-era lifecycle), and LP-075 (BLS aggregate).

Naming evolution (2026-03). LP-073 was originally titled "Corona." It now describes **Pulsar**, the Lux production fork at `github.com/luxfi/pulsar`, which inherits Corona's Sign1/Sign2/Combine math byte-equal but adds a key-era lifecycle (LSS-managed generations, verifiable resharing, activation certificate, rollback) suitable for permissionless validator-set rotation. Throughout this proof, "Corona" refers to the lattice signing primitive (the Sign1/Sign2/Combine math); "Pulsar" refers to the systems-layer composition that wraps it for dynamic permissionless deployment. Soundness results below derive from the SUF-CMA reduction of the underlying Corona signing scheme; lifecycle additions (resharing, activation, rollback) are orthogonal to the per-signature unforgeability argument and are addressed separately in LP-073's lifecycle section.

1 QuasarCert

Definition 1 (Validator set)**.** $\mathcal{V} = \{v_1, \ldots, v_n\}$, $|\mathcal{V}| = n$. *Stake distribution* $\mathrm{stk} : \mathcal{V} \to \mathbb{N}$ *with total stake* $S = \sum_i \mathrm{stk}(v_i)$. *At most a stake fraction* $f < 1/3$ *is Byzantine. Threshold* $\tau = 2/3$, *i.e. a valid quorum collects stake* $\geq \lceil 2S/3 \rceil$.

Definition 2 (Certificate subject – Def. 6.1 of LP-020)**.** *The* certificate subject *is the byte string*

$$\mathsf{subj} = \mathsf{chain_id} \,\|\, \mathsf{epoch} \,\|\, \mathsf{round} \,\|\, \mathsf{mode} \,\|\, P \,\|\, Q \,\|\, Z \,\|\, A \,\|\, B \,\|\, M \,\|\, F \,\|\, \mathsf{parent_} * \mathsf{_hashes} \,\|\, \mathsf{gas/fee}$$

where P = `pchain_validator_root`, Q = `qchain_ceremony_root`, Z = `zchain_vk_root`, A = `achain_state_root`, B = `bchain_state_root`, M = `mchain_msg_root`, F = `fchain_state_root`. *All three lanes sign or prove against* subj.

Definition 3 (QuasarCert – Def. 6.2 of LP-020)**.** *For subject* subj*, a* QuasarCert *is the triple*

$$\mathcal{C} = (\sigma_{\mathrm{BLS}}, \sigma_{\mathrm{RT}}, \pi_{\mathrm{ZK}})$$

where:

- σ_{BLS}*: BLS12-381 aggregate from validators of stake* $\geq \tau S$ *on* subj*, verified against* pchain_validator_root*;*
- σ_{RT}*: Corona two-round threshold signature aggregating shares of stake* $\geq \tau S$ *on* subj*, verified against* qchain_ceremony_root*;*
- π_{ZK}*: Z-Chain Groth16 proof that* n *ML-DSA-65 signatures on* subj *verify under the public-key vector committed by* zchain_vk_root.

$\mathsf{Verify}(\mathcal{C}) = 1$ *iff every lane verifies and every lane binds the* same subj.

Definition 4 (Hardness assumptions)**.** *We isolate one assumption per lane.*

1. ***DL/co-CDH on BLS12-381*** *(classical). Discrete log / co-CDH in* $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$*. BLS aggregate signatures with proof-of-possession are EUF-CMA under co-CDH.*
2. ***Module-LWE*** *(post-quantum).* $\mathsf{MLWE}_{q,k,\ell,\chi}$ *for the Corona / ML-DSA-65 parameter sets in App. E.*
3. ***Module-SIS*** *(post-quantum).* $\mathsf{MSIS}_{q,k,\beta}$ *for the same parameter sets. Combined with MLWE, gives EUF-CMA for ML-DSA-65 (FIPS 204) and SUF-CMA for Corona.*
4. ***KEA / AGM*** *(auxiliary). Groth16 is knowledge-sound in the algebraic group model on BLS12-381.*

2 Adversary Model

Definition 5 (Adversary)**.** *A non-uniform polynomial-time adversary* $\mathcal{A}$ *controls a subset* $F \subset \mathcal{V}$ *of validators with stake share* $< \tau$*.* $\mathcal{A}$ *is given:*

- *the public keys of all validators (BLS, Corona, ML-DSA);*
- *signing oracles for the corrupt set* F*;*
- *the public verification key* vk *for the Z-Chain Groth16 circuit;*
- *the random oracle* H *(programmable in the proof).*

We consider two variants.

$\mathcal{A}_C$ – **classical PPT.** *Standard probabilistic polynomial time.*

$\mathcal{A}_Q$ – **quantum PPT.** *Polynomial-time on a quantum Turing machine; can run Shor and Grover, has full quantum random oracle access (QROM).*

$\mathcal{A}$ wins *the soundness game if it outputs* $(\mathsf{subj}^\star, \mathcal{C}^\star)$ *such that* $\mathsf{Verify}(\mathcal{C}^\star) = 1$ *and* $\mathsf{subj}^\star$ *was not endorsed by any honest stake-weighted quorum.*

Remark 1 (Static vs adaptive)**.** *The proofs in Sections 3–7 are stated under* static *corruption. Adaptive corruption is handled in App. C via secure-erasure or programmable-RO hybrids, with at most an* $O(n \cdot Q_H)$ *multiplicative loss. All Lux validators perform Go-side memory zeroing of Corona signing randomness; HSM-backed T-Chain signing on the live network provides hardware-enforced erasures, so Cor. 21 applies tightly.*

3 Soundness

Theorem 5 (QuasarCert classical soundness)**.** *Under the assumptions of Def. 4, if* $\mathcal{C}^\star = (\sigma^\star_{\mathrm{BLS}}, \sigma^\star_{\mathrm{RT}}, \pi^\star_{\mathrm{ZK}})$ *verifies against a subject* $\mathsf{subj}^\star$ *that was not endorsed by validators of stake* $\geq \tau S$*, then* $\mathcal{A}_C$*'s advantage satisfies*

$$\mathsf{Adv}^{\mathrm{forge}}_{\mathcal{A}_C} \;\leq\; \min\!\big(\epsilon^{\mathrm{coCDH}}_{\mathrm{BLS}},\; \epsilon^{\mathrm{MLWE,MSIS}}_{\mathrm{RT}},\; \epsilon^{\mathrm{KEA}}_{\mathrm{ZK}} + n \cdot \epsilon^{\mathrm{MLWE,MSIS}}_{\mathrm{MLDSA}}\big) \;=\; \mathsf{negl}(\lambda).$$

Proof. We argue per lane and then take the conjunction.

Lane 1 (BLS). $\sigma^\star_{\mathrm{BLS}}$ is a BLS12-381 aggregate over $\mathsf{subj}^\star$. Proof-of-possession on each validator's $\mathsf{pchain_validator_root}$ entry rules out rogue-key attacks [1]. By EUF-CMA of BLS aggregate signatures under co-CDH, a valid $\sigma^\star_{\mathrm{BLS}}$ implies that each validator v_i in the aggregator set $S^\star_{\mathrm{BLS}}$ produced an individual BLS signature on $\mathsf{subj}^\star$. Stake-weighting then gives $\mathrm{stk}(S^\star_{\mathrm{BLS}}) \geq \tau S$. Forging without τ stake costs $\epsilon^{\mathrm{coCDH}}_{\mathrm{BLS}}$.

Lane 2 (Corona). $\sigma^\star_{\mathrm{RT}}$ is the output of the two-round Corona signing protocol bound to $\mathsf{qchain_ceremony_root}$ (i.e., the verification key produced by the Q-Chain DKG). By the SUF-CMA proof of Corona under MLWE/MSIS, a valid $\sigma^\star_{\mathrm{RT}}$ on $\mathsf{subj}^\star$ implies a share set $S^\star_{\mathrm{RT}}$ with $\mathrm{stk}(S^\star_{\mathrm{RT}}) \geq \tau S$. Forging costs at most $\epsilon^{\mathrm{MLWE,MSIS}}_{\mathrm{RT}}$.

Lane 3 (Z-Chain Groth16 of ML-DSA). $\pi^\star_{\mathrm{ZK}}$ proves that for the public-key vector committed by $\mathsf{zchain_vk_root}$, there exist valid ML-DSA-65 signatures $(\sigma^{\mathrm{ML}}_1, \ldots, \sigma^{\mathrm{ML}}_n)$ on $\mathsf{subj}^\star$. Knowledge-soundness in the AGM yields an extractor $\mathcal{E}$ that recovers these signatures with overwhelming probability: any deviation has advantage $\epsilon^{\mathrm{KEA}}_{\mathrm{ZK}}$. Each extracted ML-DSA-65 signature is itself EUF-CMA under MLWE/MSIS, so for any uncorrupted v_i a valid extracted signature without v_i's key costs $\epsilon^{\mathrm{MLWE,MSIS}}_{\mathrm{MLDSA}}$ (with up to n guessing loss for which validator was framed).

Conjunction. Verify is the AND of the three predicates. An attacker who breaks fewer than three lanes still has at least one lane verifying only with negligible probability. Hence $\mathsf{Adv}^{\mathrm{forge}}_{\mathcal{A}_C}$ is upper-bounded by the *minimum* of the per-lane advantages, which is negligible.

Cross-lane independence. The three bounds rely on three disjoint problems: discrete log on BLS12-381, MLWE, MSIS. No known reduction transfers a break of one to a break of another at the parameter sets of Def. 4; this is exactly the hedging argument of pq-finality-no-bls.tex §3.2. Compromise of any one of $\{\mathsf{pchain_validator_root}, \mathsf{qchain_ceremony_root}, \mathsf{zchain_vk_root}\}$ disables only the corresponding lane. □

Corollary 6 (No conflicting certificates)**.** *If* $\mathcal{C}, \mathcal{C}'$ *both verify on subjects* $\mathsf{subj}, \mathsf{subj}'$ *that share the per-height slot* $(\mathsf{chain_id}, \mathsf{round})$ *but disagree on the chain-state roots, then* $\geq S/3$ *stake is Byzantine. The slot is deliberately* epoch-blind*:* epoch *is bound inside* subj *for verification and replay protection (Thm. 14), but it is* not *part of the vote-once slot key.*

Proof. By Lane 1, $\mathrm{stk}(S_{\mathrm{BLS}}) \geq \tau S$ and $\mathrm{stk}(S'_{\mathrm{BLS}}) \geq \tau S$. Stake-weighted quorum intersection gives $\mathrm{stk}(S_{\mathrm{BLS}} \cap S'_{\mathrm{BLS}}) \geq (2\tau - 1)S = S/3$. Each honest validator signs at most one subject in each per-height slot $(\mathsf{chain_id}, \mathsf{round})$: the accept path maintains a *durable* per-height vote-once guard that is pruned strictly below the finalised tip — never while a sibling at that height could still be signed — and rejects sibling subtrees transitively, so a decided height's siblings are unsignable. Any validator in the intersection would therefore have signed two distinct subjects at the same $(\mathsf{chain_id}, \mathsf{round})$, which is detectable equivocation; if Byzantine stake is $< S/3$ this is impossible. □

Remark 2 (The slot must not key on a proposer-chosen axis)**.** *Soundness of Cor. 6 requires the vote-once slot to be* (chain_id, round) *alone. Folding a proposer-influenced value — e.g. an epoch derived from a proposer-selected validator-set height —* into *the slot key would let two honest siblings at one height carry distinct slots, so a single honest validator could sign both without equivocating; this is precisely the double-finalisation hazard. Epoch therefore lives in* subj *(bound and verified) but never in the slot. A conflicting certificate that is nonetheless observed is treated as a fatal, halting event — the BFT-safe response, since two verified certificates at one height are irrefutable evidence of* $\geq S/3$ *Byzantine stake.*

4 Finality Admission: No Double-Finalization

Corollary 6 shows that two conflicting certificates at one height imply $\geq S/3$ Byzantine stake *given* its premise: each honest validator signs at most one subject per per-height slot. That premise is exactly what the height-10 devnet storm and the 1082814 mainnet fork broke. This section formalises the durable *decided-height frontier* that restores the premise unconditionally — including across a correlated rolling restart, the window a purely in-memory guard leaves open — and proves that an observable double-finalization is unreachable below $S/3$ Byzantine stake. The construction is the mechanical analog of avalanchego's acceptance frontier (`topological.go`: a decided height's blocks are permanently unsignable via the monotone last-accepted height plus `rejectTransitively`), *lifted onto the Quasar sign path.* The canonical statements live in `definitions/finality-definitions.tex`; we restate what the proof needs.

Definition 6 (Two finality authorities)**.** *A Lux chain runs two decision machines over one block DAG: the* acceptance *machine* $\mathcal{A}$ *(local preference/accept; admits siblings and nondeterminism; never irreversible alone) and the* certified-finality *map* $\mathcal{F} =$ byHeight*, where* byHeight$[H]$ *is the canonical execution commitment finalized at* H *and is written* only *by folding a* Prism*-passing quorum certificate over the frontier-extending branch (or the bootstrap frontier-trust path). The 1082814 break was a path other than a verified-QC fold advancing* $\mathcal{F}$*; the fix makes the verified-QC fold its sole writer.*

Definition 7 (Durable decided-floor)**.** *Let* FinHeight *be the certified height (*$(0, \perp)$ *before the first cert; the* vm.LastAccepted *recovery hint is non-authoritative and never sets it —* PART-A*). Define*

$$\varphi \;=\; \max\big(\mathsf{FinHeight},\ \mathsf{fsync}(\mathsf{finalizedThrough}),\ \mathsf{height}(\mathsf{vm.LastAccepted})\big).$$

finalizedThrough *is a monotone height persisted (fsync'd) in the vote-guard file at each finalize.* φ *is monotone, restart-surviving, and (Lemma 7) a sound lower bound on the network's decided height.* ***Sign-gate only:*** φ *is read by the sign gate and never writes* byHeight*,* FinHeight*, or the equivocation index, and reads only the hint's* height*, never its identity.*

Definition 8 (Sign admissibility)**.** *An honest validator binds its single accept-signature at height* H *on canonical commitment* C *only if (i)* $H > \varphi$ *(decided-height gate), and (ii) the per-height slot at* H *is empty or already bound to* C *(idempotent); a different* C *at an undecided* H *is refused on the single-phase path, or governed by the round-scoped lock rule (one precommit per* (H, round)*; cross-round migration only on a proof-of-lock) on the view-change path.*

Definition 9 (Observable double-finalization)**.** *Two certificates* $\mathcal{C}_A, \mathcal{C}_B$ *that both verify against the active validator set at height* H*'s epoch with canonical commitments* $C_A \neq C_B$*. Certs are keyed on the* canonical execution commitment*, so two envelopes wrapping the same* C *are a duplicate alias, never this fault (Rmk. 3).*

Lemma 7 (Floor soundness)**.** $\varphi \leq \mathsf{FinHeight}_{\text{network}}$ *at all times.*

Proof. Each argument of the max is a height the network has finalized. FinHeight is certified by definition. finalizedThrough is written only by the finalizer acceptWithCertCore after a Prism-passing cert commits, and only up to the height the VM actually applied (the fail-closed accept: finality advances only to highestAccepted, never past the VM's state). vm.LastAccepted is the node's applied head, and a block is VM-applied only inside that same finalizer; a state-sync import lands only on an already-finalized frontier. Hence every argument is $\leq \mathsf{FinHeight}_{\text{network}}$, so is their maximum. φ therefore never refuses signing at a height the network has *not* decided — it costs no liveness. □

Lemma 8 (No-window durability)**.** *For every height H and at every instant on an honest node, either the in-memory slot at H is present, or $\varphi \geq H$ (or both). There is no instant at which both are absent.*

Proof. The slot at H is created when the node first signs at H and is deleted only by pruneCommittedSlotsBelow(h) with the *strict* test $k.\mathsf{Height} < h$. That prune runs inside the finalizer, in the same critical section (`slotMu`) that first advances $\varphi \leftarrow \max(\varphi, h)$ and persists both in one fsync'd write. So the slot at H can be removed only by a prune with $h > H$, and any such prune has already set $\varphi \geq h > H$. Conversely, before H is ever finalized the slot persists (no prune with $h > H$ has run). The two conditions therefore cover every instant; the removal of the slot and the raising of the floor are atomic on disk, so a crash between them is impossible. In particular the just-finalized tip's own slot is *retained* (strict, not inclusive) until a higher height finalizes — closing precisely the inclusive-prune window that admitted the height-10 storm's late sibling. □

Lemma 9 (Honest one-canonical-per-height, durable)**.** *An honest validator obeying Def. 8 binds its accept-signature to at most one canonical commitment at any height H, and this holds across process restarts.*

Proof. Fix H and suppose the validator has bound C at H. Consider any later sign request (H, C') with $C' \neq C$. By Lemma 8 at least one of two cases holds. *(a) The slot at H is present:* it is bound to C, so admissibility clause (ii) refuses C' on the single-phase path; on the view-change path the lock rule forbids a second precommit at the locked round and permits migration only on a proof-of-lock for C', which requires α honest-majority prevotes for C' and is itself unique per round (so it cannot coexist with a POL for C — the standard Tendermint lock/POL argument, sound when $2\alpha - n > f$). *(b) $\varphi \geq H$:* clause (i) refuses outright. Durability: φ is reconstructed at boot from the fsync'd finalizedThrough and vm.LastAccepted before any signing goroutine starts, so a restart cannot reset it below a height already decided (the correlated rolling-restart window that a memory-only guard leaves open). Hence at most one canonical is ever signed at H. □

Theorem 10 (No double-finalization)**.** *Assume the accept quorum requires signer stake $\geq \tau S$ with $\tau \geq 2/3$, the active validator set at H's epoch is fixed by the epoch-bound set-root cross-checked on every incoming cert, and Byzantine stake is $< S/3$. If every honest validator obeys Def. 8 with the durable floor of Def. 7, then no observable double-finalization (Def. 9) can occur at any height.*

Proof. Suppose two certs $\mathcal{C}_A, \mathcal{C}_B$ exist at height H with $C_A \neq C_B$, both verifying against the set-root of H's epoch. Since both fold the same epoch set-root (enforced by the incoming-cert cross-check: a cert whose set-root differs from the one the node recomputes at the block's P-chain epoch height is dropped), the two signer sets S_A, S_B are drawn from the *same* validator set of total stake S. Each has stake $\geq \tau S$, so by stake-weighted quorum intersection $\mathrm{stk}(S_A \cap S_B) \geq (2\tau - 1)S \geq S/3$. As Byzantine stake is $< S/3$, the intersection contains an *honest* validator v,

which then signed both C_A and C_B at height H. This contradicts Lemma 9. Hence no such pair exists. Equivalently: any observed conflicting cert pair is irrefutable evidence of $\geq S/3$ Byzantine stake — now *unconditionally*, without the "healthy sequential net" caveat the pre-fix guard silently required, because Lemmas 8–9 hold under the contested-finality storm and across restarts. □

Corollary 11 (PART-A non-interference)**.** *The durable floor never advances $\mathcal{F}$: for all reachable states,* FinHeight $= (0, \perp)$ *until the first* Prism*-passing cert folds, and thereafter* byHeight *is written only by the verified-QC fold.*

Proof. By Def. 7 the sign gate is a pure reader of φ; its only side effects are refusing a signature and (on a first bind) persisting the per-height slot — neither touches byHeight, FinHeight, or the equivocation index. The floor reads the recovery hint's height only, never its identity, so it cannot synthesise a byHeight entry. Thus the fail-safe can only ever *refuse more signing*; it cannot manufacture finality. This is the incident-1082814 PART-A invariant, preserved. □

Remark 3 (Canonical keying)**.** *Both certs in Def. 9 are compared on their inner execution commitment $C =$* CanonicalID*, not the proposervm outer envelope id. A block with no inner commitment (a bare, non-wrapped VM block) degrades to its own outer id, so the scheme is inert there and reduces exactly to pre-fix outer-id behaviour. Two outer envelopes wrapping the same C (the literal 1082814 evidence: $A = 2U2pR3D$, $B = wDMUyGy$ both wrapping inner $5DEgMudU$) are a* duplicate alias*, and the equivocation detector — which now runs the full verify-before-slash predicate before recording any evidence — never fires on them; only a verified cert over a* different C *is recorded as a* DoubleVote*, and the node rejects the second cert and keeps running rather than halting the fleet.*

Remark 4 (Empirical validation)**.** *The construction was validated on a 5-node contested-finality storm (node v1.34.6, consensus v1.35.16): concurrent multi-proposer building drove certified finality past the height-10 fault point to height 76 with exactly one canonical commitment per height fleet-wide, zero* reportCertEquivocation *events, byte-identical 5/5 convergence, and a DEX* 0x9999 *settlement fill at block 74. A rolling-restart* mid-storm *reconverged cleanly, exercising the durable cross-restart floor of Lemma 9. The in-repo regression is* `engine/chain/finalize_resign_test.go` *(*`TestFinalizeThenResign_DecidedHeightIsUnsignable`*), which fails on the inclusive-prune / no-gate code and passes on the shipped fix, plus the* `storm_convergence_test.go` *safety assertion with the former* `-race` *skip removed.*

5 Parallel Lane Liveness

Theorem 12 (Parallel-lane liveness)**.** *The three lanes of QuasarCert can be aggregated independently and in parallel. Specifically, let $T_{\mathrm{BLS}}, T_{\mathrm{RT}}, T_{\mathrm{ZK}}$ be the wall-clock durations of the three lane aggregations under partial synchrony with bounded delay Δ and honest stake $\geq \tau$. Then there exists an honest schedule that produces $\mathcal{C}$ in time*

$$T_{\mathrm{cert}} \;\leq\; \max\bigl(T_{\mathrm{BLS}},\, T_{\mathrm{RT}},\, T_{\mathrm{ZK}}\bigr) + \Delta.$$

Proof sketch. The three aggregations communicate only through the shared subj. Once subj is fixed for a round (which requires only the chain-state roots, all available at round start), each lane runs a separate aggregation network: BLS aggregator on the P-Chain validator network; Corona two-round protocol on the Q-Chain DKG mesh; Z-Chain Groth16 prover (epoch-amortised, asynchronous to block production). Honest validators contribute to all lanes they hold keys for; under stake $\geq \tau$

each lane independently completes within its own bound. The certificate is finalised when the last lane lands; the extra Δ accounts for one round of network propagation to combine the three artifacts.

The Z-Chain proof, in particular, is generated by the Z-Chain prover service over an epoch window and lands asynchronously: the epoch's π_{ZK} becomes part of every QuasarCert in that epoch once posted. This is the LP-020 Section 6.4 "epoch ZK landing" mechanism and does not block per-block finality of the BLS+Corona core. □

6 Post-Quantum Safety

Theorem 13 (Post-quantum safety)**.** *Define the standalone PQ predicate* $\mathsf{VerifyPQ}(\mathcal{C}) = \mathsf{RT.Verify}(\sigma_{\mathrm{RT}}) \wedge \mathsf{ZK.Verify}(\pi_{\mathrm{ZK}})$*. Against a quantum adversary* $\mathcal{A}_Q$*,*

$$\mathsf{Adv}^{\mathsf{VerifyPQ}}_{\mathcal{A}_Q} \;\le\; 2 \cdot \max\!\big(\epsilon^{Q}_{\mathrm{MLWE}},\, \epsilon^{Q}_{\mathrm{MSIS}}\big) \;=\; \mathsf{negl}(\lambda).$$

That is: even if $\mathcal{A}_Q$ *trivially breaks BLS via Shor, the PQ-only predicate remains sound.*

Proof. $\mathcal{A}_Q$ must produce $\sigma^{\star}_{\mathrm{RT}}$ that verifies against $\mathsf{qchain_ceremony_root}$ and $\pi^{\star}_{\mathrm{ZK}}$ that verifies against $\mathsf{zchain_vk_root}$.

Forging σ_{RT} *quantumly* requires breaking MLWE or MSIS at the Corona dimension $d = 2048$, modulus $\log_2 q = 48$ (App. E). The best known quantum cost is BDGL sieving with Grover speedup, giving $T_{\mathrm{quantum}} \ge 2^{130}$ operations.

Forging π_{ZK} *quantumly* is delicate: Groth16's classical pairing-based knowledge-soundness reduces to discrete log on BLS12-381, which is broken by Shor. In the post-quantum setting we fall back to the soundness of the underlying ML-DSA-65 signatures it proves. A quantum forger of π_{ZK} either (a) extracts no real witness and yet the proof verifies (breaks knowledge-soundness in the AGM, classical only – not assumed against $\mathcal{A}_Q$), or (b) extracts ML-DSA-65 signatures the adversary did not legitimately hold. Case (b) costs $\epsilon^{Q}_{\mathrm{MLDSA}} \ge 2^{128}$ at ML-DSA-65 (NIST Cat. 3) per signature, with at most n guessing loss. The standalone-PQ guarantee for π_{ZK} thus reduces to the PQ unforgeability of ML-DSA-65 signatures committed by $\mathsf{zchain_vk_root}$.

A guessing reduction picks the targeted lane with probability 1/2 and inherits the corresponding bound; combined with negl per lane this gives the stated Adv.

Future-work caveat. Replacing Groth16 with a STARK-of-pairing or hash-based SNARK eliminates the dependence on classical knowledge-soundness in π_{ZK}. This is tracked as the "post-quantum SNARK swap" in LP-020 §11 and is out of scope for 3.0 final. □

Remark 5 (Combined three-lane PQ guarantee)**.** *The full QuasarCert remains sound against* $\mathcal{A}_Q$ *as long as* at least one *of* {*Corona, ML-DSA path*} *survives. Equivalently: the survival predicate is "1 of 2 PQ pillars". This matches the pq-finality-no-bls.tex Proposition (Hedged PQ security) under the conjunction "MLWE hard* $\vee$ *MSIS hard," and is the operative claim for the 2025-12-25 activation.*

7 Subject-Binding Replay Protection

This section is new in 3.0 final. The result formalises why the *certificate subject* (Def. 2) makes a valid QuasarCert from epoch e_1 unusable at any other epoch $e_2 \neq e_1$.

Theorem 14 (Subject-binding replay protection)**.** *Let $\mathcal{C}$ be a QuasarCert that verifies for subject* subj_1 *at epoch* e_1*. Let* subj_2 *differ from* subj_1 *in any field of Def. 2. Then the probability that $\mathcal{C}$ also verifies for* subj_2 *under the assumptions of Def. 4 is* $\mathsf{negl}(\lambda)$*. In particular, the cert cannot be replayed across distinct* epoch *values (or distinct* chain_id*,* round*,* mode*, or any chain root in the subject).*

Proof. We show that each lane is collision-resistant on the subject input.

BLS. σ_{BLS} is computed as $\prod_i H_{\mathbb{G}_2}(\mathsf{subj})^{sk_i}$ where $H_{\mathbb{G}_2}$ is a hash-to-curve. If σ_{BLS} verifies for both subj_1 and $\mathsf{subj}_2 \neq \mathsf{subj}_1$, then there exists σ_{BLS} such that $e(\sigma_{\mathrm{BLS}}, g_1) = e(H_{\mathbb{G}_2}(\mathsf{subj}_1), \mathsf{apk}) = e(H_{\mathbb{G}_2}(\mathsf{subj}_2), \mathsf{apk})$. The pairing equality forces $H_{\mathbb{G}_2}(\mathsf{subj}_1) = H_{\mathbb{G}_2}(\mathsf{subj}_2)$ (in $\mathbb{G}_2$), which is a hash-to-curve collision and hence negligible.

Corona. The Corona challenge is $c = H(\mathbf{w}, \mathsf{subj})$. If σ_{RT} verifies for both subjects with the same commitment $\mathbf{w}$, then $H(\mathbf{w}, \mathsf{subj}_1) = H(\mathbf{w}, \mathsf{subj}_2)$, again a hash collision. If the commitments differ, this is a fresh forgery and reduces to MLWE/MSIS.

Groth16. π_{ZK} encodes subj as a public input bound at SRS preprocessing. Verifying the same proof against a different public input with non-negligible probability contradicts knowledge-soundness in the AGM (the verifier equation $e(A, B) = e(\alpha, \beta) \cdot e(\sum_i \mathsf{subj}[i] \cdot \mathsf{IC}_i, \gamma) \cdot e(C, \delta)$ is linear in subj and pinning a single proof to two distinct linear combinations is a collision in the AGM extractor).

Each lane's replay probability is therefore $\mathsf{negl}(\lambda)$; since Verify is the AND of the three, replay across distinct subjects has at most the minimum of these advantages, which remains negligible. □

Remark 6 (Concrete fields blocked)**.** *Because* subj *contains* epoch*,* round*, and* chain_id *explicitly, replay across epochs, across rounds within an epoch, or across chains (e.g., devnet to mainnet) is ruled out individually. The chain-root fields (P, Q, Z, A, B, M, F) further block replay against any state divergence: even if two distinct epochs share epoch number (impossible by P-Chain rules but treated formally), the $A/B/F$ roots disagree as soon as any inclusion-list transaction differs.*

8 GPU Verifier Refinement (Informal)

LP-132 specifies the QuasarGPU adapter. Its core entry point is `drain_cert_lane(lane, batch)` which produces a verdict $\{\mathsf{accept}, \mathsf{reject}, \mathsf{abstain}\}$ for each cert in batch. The CPU reference is a sequential loop calling the per-lane verifier.

Cross-backend equivalence (LP-132 invariant). Let V_{cpu} and V_{gpu} denote the verdict functions. LP-132 requires

$$V_{\mathrm{gpu}}(\pi(\mathsf{batch})) = \pi(V_{\mathrm{cpu}}(\mathsf{batch})) \quad \text{for any canonical-tx-index permutation } \pi.$$

That is, the GPU implementation differs from CPU only by deterministic reordering. The reordering is induced by per-warp scheduling, which the adapter resolves at output by sorting on the canonical tx index.

Argument. Each lane verifier is a pure function on (subj, lane artifact) with no shared state across batch elements. GPU parallelism replicates this function across threads with thread-local memory; the only global state read is the verification key roots (read-only). Hence each per-element verdict is identical to the CPU verdict. The output ordering is canonicalised by sorting on the tx index, eliminating the only nondeterminism. By inspection of the LP-132 reference loop, the GPU

adapter produces verdict-equivalent output up to permutation, and the canonical sort removes the permutation.

What this is not. This is an *informal* argument, not a machine-checked proof. A machine-checked refinement would require reasoning about CUDA / HIP memory models and is tracked in the `lean/Consensus/` GPU refinement task. The cross-backend differential test in LP-132 `tests/cross_backend_test.go` provides empirical evidence at the engineering boundary.

9 Pulsar Key-Era Lifecycle Lemmas

This section formalises the lifecycle properties of LP-073 Pulsar that sit *above* the per-signature SUF-CMA argument. The Corona kernel of Pulsar is treated as an oracle that produces a signature σ_{RT} verifying under $\mathsf{GK} = (\mathbf{A}, \tilde{\mathbf{b}})$ given $\geq t$ shares $\{\mathbf{s}_i\}$ that satisfy the Lagrange-recombination identity for the master secret $\mathbf{s}$. We prove four lemmas that together formalise the "persistent body, rotating beam" property of a key era.

Definition 10 (Pulsar share state)**.** *A Pulsar share state at validator v is a tuple*

$$\pi_v = \big(\eta,\ g,\ r,\ V,\ t,\ \mathbf{A},\ \tilde{\mathbf{b}},\ \mathbf{s}_v,\ \boldsymbol{\lambda}_v,\ \mathsf{seeds}_v,\ \mathsf{macKeys}_v\big)$$

where $\eta \in \mathbb{N}$ is the KeyEraID*, $g \in \mathbb{N}$ the* Generation*, $r \in \mathbb{N}$ the* RollbackFrom *counter ($r = 0$ on forward transitions), $V \subseteq \mathcal{V}$ the active validator subset of size $|V| = K$, t the threshold, $\mathbf{A} \in R_q^{m_{\mathrm{pk}} \times n_{\mathrm{vec}}}$ and $\tilde{\mathbf{b}} \in R_\xi^{m_{\mathrm{pk}}}$ the persistent group-key components, $\mathbf{s}_v$ the share assigned to v, $\boldsymbol{\lambda}_v$ the precomputed Lagrange coefficient for v in V, and $\mathsf{seeds}_v, \mathsf{macKeys}_v$ the pairwise PRF seeds and BLAKE3 MAC keys for v vs. every $w \in V \setminus \{v\}$.*

Definition 11 (GroupKey hash)**.** *The* GroupKey hash $\mathsf{GKH}(\pi_v)$ *is the 32-byte digest*

$$\mathsf{GKH}(\pi_v) = \mathsf{BLAKE3}\big(\mathsf{WriteTo}(\mathbf{A}) \,\|\, \mathsf{WriteTo}(\tilde{\mathbf{b}})\big)[:32].$$

$\mathsf{WriteTo}$ *is the LP-073 §4 wire serialisation: little-endian* `uint64` *coefficient stream with 8/16-byte length prefixes for* Vector*/*Matrix*. Two share states π_v, π_w* share a GroupKey *iff* $\mathsf{GKH}(\pi_v) = \mathsf{GKH}(\pi_w)$.

Definition 12 (Lifecycle transitions)**.** *Let Π denote a global Pulsar state $(\pi_v)_{v \in V}$. We distinguish three transitions:*

- ***Refresh*** *($\Pi \xrightarrow{\mathrm{Refresh}} \Pi'$): HJKY97 zero-polynomial proactive refresh, same V, t, same $(\mathbf{A}, \tilde{\mathbf{b}})$, share distribution rotates.*
- ***Reshare*** *($\Pi \xrightarrow{\mathrm{Reshare}(V', t')} \Pi'$): Desmedt–Jajodia97 redistribution to validator subset V' with threshold t'; same $(\mathbf{A}, \tilde{\mathbf{b}})$.*
- ***Reanchor*** *($\Pi \xrightarrow{\mathrm{Reanchor}} \Pi'$): fresh ceremony or DKG produces a new $(\mathbf{A}', \tilde{\mathbf{b}}')$ and bumps η.*

Lemma 15 (Generation monotonicity)**.** *For every Pulsar transition $\Pi \to \Pi'$ at fixed η and any $v \in V \cap V'$, $g(\pi'_v) = g(\pi_v) + 1$.*

Proof. The LSS coordinator (LP-073 §9 (LSS-Pulsar)) advances g by one on each successful Refresh or Reshare. Failed activation triggers ROLLBACK; we treat ROLLBACK(g_{target}) as a separate transition $\Pi \to \Pi'$ for which the post-state has $g(\pi'_v) = g(\pi_v) + 1$ *and* $r(\pi'_v) = g_{\mathrm{target}}$. This preserves monotonicity — g is bumped on every transition, including reverts — while marking lineage via r. The activation certificate (Theorem 19 below) carries g as an explicit field,

so any verifier can audit monotonicity from the on-chain record. The Lean encoding of this property appears as `reshare_threshold_maintained` and the implicit chain of generations in `Crypto.Threshold.Reshare` of the canonical proof tree at `lean/Crypto/Threshold/Reshare.lean`. □

Lemma 16 (KeyEraID stability across Reshare)**.** *For every* $\Pi \xrightarrow{\text{Reshare}(V',t')} \Pi'$ *and every* $v \in V'$, $\eta(\pi'_v) = \eta(\pi_v)$ *for any* $v \in V \cap V'$ *(or* $\eta(\pi_v)$ *inherited from the launch transition for new* $v \in V' \setminus V$*).* η *bumps only at Reanchor.*

Proof. Reshare carries $(\mathbf{A}, \tilde{\mathbf{b}})$ across by construction (Desmedt–Jajodia identity, Lemma 17 below); η is the chain-state pointer to the GroupKey lineage and is updated only when the underlying $(\mathbf{A}, \tilde{\mathbf{b}})$ changes, i.e., at Reanchor. The LSS adapter contract (LP-073 §9 (LSS-Pulsar).3) explicitly preserves η across the call `DynamicResharePulsar`; the LP-073 acceptance test `TestKeyEraIDPreservedAcrossReshar` (`threshold/protocols/lss/lss_pulsar_test.go`) verifies this property at the bytes-on-disk level for the Go canonical. □

Lemma 17 (GroupKey byte-identity across Refresh and Reshare)**.** *For every* $\Pi \to \Pi'$ *that is either Refresh or* Reshare(V', t')*, and any* $v \in V'$*,*

$$\mathsf{GKH}(\pi'_v) = \mathsf{GKH}(\pi_w) \quad \forall w \in V \cap V'.$$

That is, the BLAKE3 digest of the wire-serialised $(\mathbf{A}, \tilde{\mathbf{b}})$ *is identical pre- and post-transition, not merely structurally equal.*

Proof. Refresh and Reshare update only the share polynomial; neither alters the public matrix $\mathbf{A}$ nor the rounded public-key vector $\tilde{\mathbf{b}}$. We verify the Lagrange-recombination identity that underwrites this preservation. Let $f(x) \in R_q[X]$ be the old-set sharing polynomial with $f(0) = \mathbf{s}$ and $f(\alpha_v) = \mathbf{s}_v$ for $v \in V$. After Reshare(V', t'), the new polynomial is constructed as

$$G(x) = \sum_{i \in Q} \lambda_i^Q(0) \cdot g_i(x), \qquad g_i(0) = \mathbf{s}_i,\ \deg g_i = t' - 1,$$

where $Q \subseteq V$ is a qualified old subset of size $\geq t_{\text{old}}$ and $\lambda_i^Q(0)$ are the Lagrange coefficients of Q evaluated at 0. Then

$$G(0) \;=\; \sum_{i \in Q} \lambda_i^Q(0) \cdot g_i(0) \;=\; \sum_{i \in Q} \lambda_i^Q(0) \cdot \mathbf{s}_i \;=\; \sum_{i \in Q} \lambda_i^Q(0) \cdot f(\alpha_i) \;=\; f(0) \;=\; \mathbf{s}, \tag{1}$$

where the penultimate equality is the standard Lagrange interpolation identity over R_q, applied componentwise to the polynomial coefficients. Hence the master secret $\mathbf{s}$ is preserved, the LWE error $\mathbf{e}$ is unchanged (it was sampled at genesis to form $\tilde{\mathbf{b}}$ and erased with the dealer state; signers only need shares of $\mathbf{s}$, never $\mathbf{e}$), and the public-key vector $\tilde{\mathbf{b}} = \lfloor \mathbf{A}\mathbf{s} + \mathbf{e} \rceil_\xi$ remains unchanged because both $(\mathbf{A}, \mathbf{s}, \mathbf{e})$ remain unchanged. WriteTo is a deterministic byte function of $(\mathbf{A}, \tilde{\mathbf{b}})$, so the BLAKE3 digest is identical. For Refresh, an analogous argument applies with $z_i(x)$ a zero-polynomial of degree $t - 1$ [21]: the constant term contributes nothing to $G(0)$, so $\mathbf{s}$, $\mathbf{e}$, and hence $\tilde{\mathbf{b}}$ are byte-preserved.

The identity (1) is mechanised in `lean/Crypto/Threshold/Reshare.lean` as the axiom `reshare_same_secret` together with theorem `reshare_preserves_public_key`, which derives pubkey_from_shares(newShares, t') = pubkey(key) from `reshare_same_secret`. □

Lemma 18 (Rollback-lineage forward reconstruction)**.** *Let* $\Pi_0 \xrightarrow{\text{Reshare}} \Pi_1 \xrightarrow{\text{Reshare}} \cdots \xrightarrow{\text{Reshare}} \Pi_k$ *be a chain of forward transitions at fixed* η*, and suppose* $\Pi_k \xrightarrow{\text{Rollback}(j)} \Pi_{k+1}$ *for some* $j < k$*. Then the chain* $(\eta_i, g_i, r_i)_{i=0}^{k+1}$ *uniquely reconstructs the forward-only path* $\Pi_0 \to \Pi_1 \to \cdots \to \Pi_j$ *that the rollback restored to.*

Proof. Each transition records (η_i, g_i, r_i) on chain. By Lemma 15, g_i is strictly monotone in i. By construction, $r_i = 0$ iff transition i is forward; $r_i = j > 0$ iff transition i is a rollback to generation j. To reconstruct the forward-only path: walk the chain in reverse, skipping any transition i with $r_i > 0$ together with all transitions $i' > j$ such that $g_{i'} > j$ and $r_{i'} = 0$ for $i' \le i$ (these are the reverted forward steps). The remaining sequence is the canonical forward lineage of any state restored to. Uniqueness follows from the strict monotonicity of g within an era (Lemma 15) and the fact that r unambiguously labels rollback targets. □

Theorem 19 (Activation-certificate soundness)**.** *Let* $\mathsf{ActivationMsg}_{\eta,g+1}$ *be the wire payload of LP-073 §8.3 (activation cert), binding the new generation* $g+1$ *at fixed era* η*. The verifier* $\mathsf{VerifyActivation}(\Pi, \Pi', \sigma)$ *accepts iff* σ *is a valid Pulsar threshold signature on* $\mathsf{ActivationMsg}_{\eta,g+1}$ *that verifies under the* unchanged *GroupKey* $(\mathbf{A}, \tilde{\mathbf{b}})$ *of* Π*. Then under MLWE/MSIS at the LP-073 §2 parameters,*

$$\Pr\big[\mathsf{VerifyActivation} = 1 \wedge \pi'_v \text{ does not satisfy (1)}\big] \le \epsilon_{\mathrm{RT}}^{\mathrm{MLWE,MSIS}} = \mathsf{negl}(\lambda).$$

That is: VerifyActivation *accepts iff the new committee's shares can sign under the unchanged GroupKey, which by SUF-CMA reduces to the new shares satisfying the Lagrange-recombination identity* (1)*.*

Proof. Forward direction. If Π' satisfies (1), then by Lemma 17 the GroupKey is byte-identical, and the honest threshold signing protocol (Corona Round 1 / Round 2) on the new shares produces a σ that verifies under the same $(\mathbf{A}, \tilde{\mathbf{b}})$. VerifyActivation accepts.

Reverse direction. If VerifyActivation accepts, then by the SUF-CMA reduction of Corona under MLWE/MSIS (Theorem 5.1 of LP-073 below, i.e. Theorem 5 restricted to the Corona lane), there exists a share set $\{\mathbf{s}'_v\}_{v \in V'}$ of stake $\ge t'$ that produced the signature, and these shares jointly recombine to the master secret $\mathbf{s}$ via Lagrange interpolation. By the verifier equation $Az - bc$ and the L_2 bound, the recovered $\mathbf{s}$ is consistent with $(\mathbf{A}, \tilde{\mathbf{b}})$ — otherwise the forger has produced a fresh MLWE/MSIS instance, an event of probability at most $\epsilon_{\mathrm{RT}}^{\mathrm{MLWE,MSIS}}$. Hence the new shares satisfy (1) except with negligible probability. □

Remark 7 (What activation does *not* prove)**.** VerifyActivation *is necessary but not sufficient for full Byzantine attribution: it certifies that the new committee can sign, not which old or new party (if any) sent a malformed contribution. Attribution is the responsibility of the complaint and disqualification pipeline (lattice Pedersen commit verification, LP-073 §8.5 (Pedersen DKG over* R_q*)). Failed activation triggers the non-punitive rollback ladder of LP-073 §8.4 (rollback); safety and liveness do not depend on slashing-based attribution.*

10 Gaps and Future Work

The 3.0 final scope leaves three gaps explicitly open for future work, each tracked in LP-020.

1. **Threshold ML-DSA.** Not used in 3.0 due to a rejection-sampling circularity (see App. A). Resolved-or-rejected once the Celi–del Pino–Espitau–Niot–Prest construction is stable.

2. **Adaptive corruption without erasures.** Tightness of the $O(n \cdot Q_H)$ loss in App. C; potential gain from a Fischlin-style transform.
3. **Post-quantum SNARK.** Replacing Groth16 with a STARK- or hash-based SNARK to eliminate the conditional on classical knowledge-soundness in Theorem 13.

References

[1] D. Boneh, B. Lynn, H. Shacham, "Short Signatures from the Weil Pairing," *ASIACRYPT*, 2001.

[2] Z. Kelling, V. Seesahai, "Corona: Lattice-Based PQ Threshold Signatures for Blockchain Consensus," Lux Industries, 2024.

[3] NIST, "FIPS 204: ML-DSA (Module-Lattice-Based Digital Signature Algorithm)," 2024.

[4] J. Groth, "On the Size of Pairing-Based Non-Interactive Arguments," *EUROCRYPT*, 2016.

[5] O. Regev, "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography," *J. ACM*, 56(6), 2009.

[6] R. Canetti, "Universally Composable Security: A New Paradigm for Cryptographic Protocols," *FOCS*, 2001.

[7] J. Garay, A. Kiayias, N. Leonardos, "The Bitcoin Backbone Protocol: Analysis and Applications," *EUROCRYPT*, 2015.

[8] M. Fischlin, "Communication-Efficient Non-Interactive Proofs of Knowledge with Online Extractors," *CRYPTO*, 2005.

[9] S. Bowe, A. Gabizon, I. Miers, "Scalable Multi-Party Computation for zk-SNARK Parameters in the Random Beacon Model," IACR ePrint 2017/1050, 2017.

[10] A. Gabizon, Z. J. Williamson, O. Ciobotaru, "PLONK: Permutations over Lagrange-Bases for Oecumenical Noninteractive Arguments of Knowledge," IACR ePrint 2019/953.

[11] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, N. Ward, "Marlin: Preprocessing zk-SNARKs with Universal and Updatable SRS," *EUROCRYPT*, 2020.

[12] A. Becker, L. Ducas, N. Gama, T. Laarhoven, "New Directions in Nearest Neighbor Searching with Applications to Lattice Sieving," *SODA*, 2016.

[13] NIST, "Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process," NISTIR 8413, 2022.

[14] T. Espitau, M. Tibouchi, A. Wallet, "Practical Fiat-Shamir with Aborts Threshold Signatures from Lattices," IACR ePrint, 2024.

[15] T. Laarhoven, "Sieving for Shortest Vectors in Lattices Using Angular Locality-Sensitive Hashing," *CRYPTO*, 2015.

[16] M. R. Albrecht, R. Player, S. Scott, "On the Concrete Hardness of Learning with Errors," *J. Math. Cryptol.*, 9(3):169–203, 2015.

[17] NIST, "FIPS 204: Module-Lattice-Based Digital Signature Algorithm (ML-DSA)," August 2024. Algorithm 3 (ML-DSA.Verify).

[18] A. Celi, R. del Pino, T. Espitau, G. Niot, T. Prest, "Efficient Threshold ML-DSA," USENIX Security 2025.

[19] "LP-020: Quasar Consensus," Lux Industries, 2025.

[20] "LP-132: QuasarGPU Adapter," Lux Industries, 2025.

[21] A. Herzberg, S. Jarecki, H. Krawczyk, M. Yung, "Proactive Secret Sharing or: How to Cope With Perpetual Leakage," *CRYPTO*, 1995, updated 1997.

[22] Y. Desmedt, S. Jajodia, "Redistributing Secret Shares to New Access Structures and Its Applications," Technical Report ISSE TR-97-01, George Mason University, 1997.

[23] T. M. Wong, C. Wang, J. M. Wing, "Verifiable Secret Redistribution for Archive Systems," *IEEE SISW*, 2002.

[24] "LP-073: Pulsar — Dynamic Lattice Threshold Signatures for Permissionless Validator Sets," Lux Industries, 2026.

A Threshold ML-DSA Rejection in 3.0

3.0 final does not adopt threshold ML-DSA. This appendix records why, because the trade-off recurs in every audit cycle.

A.1 The rejection-sampling circularity

ML-DSA-65 (FIPS 204, Algorithm 2) signs by repeatedly sampling $\mathbf{y} \leftarrow [-\gamma_1, \gamma_1]^\ell$, computing $\mathbf{w} = \mathbf{A}\mathbf{y}$, deriving challenge $c = H(\mu, \mathbf{w}_1)$, then computing $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ and *rejecting* the trial if $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ or $\|\text{LowBits}(\mathbf{w} - c\mathbf{s}_2)\|_\infty \geq \gamma_2 - \beta$. The rejection probability is non-negligible (roughly $1 - 1/e$ for the $\mathbf{z}$ check), so the signer iterates until accepted.

Threshold ML-DSA must produce a signature whose distribution is identical to centralised ML-DSA, including the rejection condition applied to the *aggregate* $\mathbf{z} = \sum_i \mathbf{z}_i$. Each share $\mathbf{z}_i = \mathbf{y}_i + c\mathbf{s}_{1,i}$ is a linear function of the per-share randomness $\mathbf{y}_i$. The challenge c is a function of the aggregate commitment $\mathbf{w} = \sum_i \mathbf{w}_i$, which fixes $\mathbf{y} = \sum_i \mathbf{y}_i$.

The circularity:

- Each signer must commit $\mathbf{w}_i$ *before* seeing c (otherwise it is not a Fiat–Shamir signature).
- The acceptance probability of the resulting aggregate $\mathbf{z}$ depends on the joint distribution of all $\mathbf{y}_i$, which the protocol cannot influence post-commit.
- Naive aggregation either (a) leaks information about each signer's $\mathbf{s}_{1,i}$ on rejection (because rejection is a function of $\mathbf{z}_i$ and a partial c), or (b) requires a fresh round per rejection, blowing up the round complexity by an expected factor of e and breaking the two-round target.

The Celi–del Pino–Espitau–Niot–Prest construction (USENIX Security 2025 [18]) addresses this with masked shares and amortised rejection, but the proof is in the algebraic group model with non-trivial trapdoor assumptions and is not yet audit-clean. 3.0 freezes on the alternative below.

A.2 The 3.0 alternative

Instead of threshold ML-DSA we use:

- *Per-validator individual* ML-DSA-65 signatures (no threshold, no aggregation, no rejection circularity).
- A Z-Chain Groth16 rollup proof that all n such signatures verify against $\mathsf{zchain_vk_root}$ on a common subj.

This preserves the security target (each validator's signature is EUF-CMA under MLWE/MSIS, the rollup is knowledge-sound under KEA in the AGM) without the circularity. The cost is an $n\times$ blow-up in witness size, which is precisely what Groth16 amortises across validators. See App. B for the constraint count.

B ML-DSA-65 Groth16 Circuit Analysis

This appendix derives the R1CS constraint count for verifying a single ML-DSA-65 signature inside a Groth16 circuit, establishes the circuit-independent proof size, and corrects the prover-time estimate.

B.1 ML-DSA-65 Verification Steps

ML-DSA-65 verification (Algorithm 3 of FIPS 204 [17]) performs the following on input (pk, μ, σ) where $pk = (\rho, t_1)$ and $\sigma = (\tilde{c}, z, h)$:

1. Expand $\mathbf{A} \in R_q^{k\times\ell}$ from ρ via SHAKE-128 ($k = 6$, $\ell = 5$, $q = 8\,380\,417 = 2^{23} - 2^{13} + 1$).
2. Compute $c = \mathsf{SampleInBall}(\tilde{c})$: a polynomial in R_q with coefficients in $\{-1, 0, 1\}$, Hamming weight $\tau = 49$.
3. Compute $\mathbf{w}' = \mathbf{Az} - c \cdot t_1 \cdot 2^d$ via NTT-domain arithmetic.
4. Extract high bits: $\mathbf{w}_1' = \mathsf{UseHint}(h, \mathbf{w}')$.
5. Check $\|z\|_\infty < \gamma_1 - \beta$ where $\gamma_1 = 2^{19}$, $\beta = 196$.
6. Recompute $\tilde{c}' = H(\mu, \mathbf{w}_1')$ and check $\tilde{c}' = \tilde{c}$.

B.2 R1CS Constraint Count

Each polynomial in $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ has 256 coefficients modulo q; each coefficient fits in 23 bits.

NTT. A 256-point NTT over $\mathbb{Z}_q$ requires $256 \cdot \log_2(256)/2 = 256 \cdot 4 = 1\,024$ butterfly operations. Each butterfly is one multiplication and two additions in $\mathbb{Z}_q$. In R1CS, multiplication modulo q costs 1 constraint (the multiplication gate) plus $O(1)$ range-check constraints to enforce that the result is reduced modulo q. Using standard range-check decomposition into 23-bit limbs, each $\bmod q$ reduction costs ≈ 3 R1CS constraints. Total per NTT: $\approx 1\,024 \times 4 = 4\,096$ constraints.

Matrix-vector product Az. This is $k \times \ell = 6 \times 5 = 30$ polynomial multiplications in R_q. Each polynomial multiplication via NTT requires: forward NTT of both operands (2 NTTs), pointwise multiply (256 multiplications), inverse NTT of the result (1 NTT). However, $\mathbf{A}$ can be precomputed in NTT domain as a public input, saving $k \times \ell = 30$ forward NTTs. Remaining: 5 NTTs for $\mathbf{z}$, 30 pointwise multiplies (each 256 R1CS multiplications + mod-q reductions), 6 inverse NTTs for the k result polynomials, plus 5 additions of ℓ-way sums.

- Forward NTT of $\mathbf{z}$: $5 \times 4\,096 = 20\,480$

- Pointwise multiply: $30 \times 256 \times 4 = 30\,720$
- Inverse NTT of results: $6 \times 4\,096 = 24\,576$
- Additions (free in R1CS): 0

Subtotal: $\approx 75\,776$ constraints.

Scalar-polynomial product $c \cdot t_1 \cdot 2^d$. The challenge c has weight $\tau = 49$, so $c \cdot t_1$ involves a sparse convolution. In the worst case this is done via NTT: $2 + 1 = 3$ NTTs plus 256 pointwise multiplies. Cost: $3 \times 4\,096 + 256 \times 4 = 13\,312$ constraints. Repeated for each of the $k = 6$ polynomials in t_1: $\approx 79\,872$. (Optimization: exploit sparsity of c to reduce to $\approx 49 \times 256 \times k$ multiply-add operations $\approx 75\,264$.)

Norm check, UseHint, hash. Norm check $\|z\|_\infty < \gamma_1 - \beta$: range check on $5 \times 256 = 1\,280$ coefficients, each ≈ 20 constraints for a 19-bit comparison. Cost: $\approx 25\,600$. UseHint: bit extraction on 6×256 coefficients, $\approx 6 \times 256 \times 10 = 15\,360$ constraints. SHAKE / hash re-computation: SHAKE-256 on $\sim$1 KB input, $\approx 25\,000$ constraints (Keccak-f[1600] at $\approx 12\,500$ constraints per permutation, 2 calls).

SHAKE-128 for A expansion. Expanding $\mathbf{A}$ from ρ requires $k \times \ell = 30$ rejection-sampling streams of SHAKE-128. Each stream produces ~ 256 coefficients in $\mathbb{Z}_q$ via rejection sampling. This is the most expensive part: $\approx 30 \times 3 \times 12\,500 = 1\,125\,000$ constraints (3 Keccak permutations per stream on average due to rejection).

Total for one ML-DSA-65 verification.

Sub-circuit	**Constraints**
A expansion (SHAKE-128)	$\sim 1\,125\,000$
Az product (NTT)	$\sim 75\,776$
$c \cdot t_1$ product	$\sim 79\,872$
Norm check	$\sim 25\,600$
UseHint	$\sim 15\,360$
Hash recomputation (SHAKE-256)	$\sim 25\,000$
Total (1 verification)	$\sim 1.35 \times 10^6 \approx 2^{22.5}$

For n validators, the circuit verifies n signatures: $C \approx n \times 1.35 \times 10^6$. For $n = 21$ (Quasar mainnet): $C \approx 2.84 \times 10^7 \approx 2^{24.8}$.

Optimisation – amortised case. If $\mathbf{A}$ is shared across all n validators (same ρ), the expansion is done once: $C \approx 1.125 \times 10^6 + n \times 2.21 \times 10^5 \approx 5.77 \times 10^6 \approx 2^{22.5}$ for $n = 21$. This is the figure used in the main text and in LP-020 §6.4. Equivalently, the per-validator amortised constraint count is $\sim 2^{20}$.

B.3 Proof Size

Groth16 proof size is circuit-independent [4]: two elements of $\mathbb{G}_1$ and one element of $\mathbb{G}_2$ over BLS12-381:

$$|\pi| = 2 \times 48 + 1 \times 96 = 192 \text{ bytes}$$

This holds regardless of the constraint count C. The 192-byte claim in the main text (Def. 6.2 of LP-020) is correct.

B.4 Prover Time

Groth16 proving requires two multi-scalar exponentiations of size C and an FFT of size C. The dominant cost is the MSM: $O(C/\log C)$ group operations in $\mathbb{G}_1$. On BLS12-381, one $\mathbb{G}_1$ scalar multiplication takes $\sim 0.5\,\text{ms}$ on a single CPU core (Apple M1 class).

For $C \approx 2^{22.5}$ (optimised, $n = 21$):

- MSM via Pippenger: $C/\log_2 C \approx 2^{22.5}/22.5 \approx 2.6 \times 10^5$ group additions.
- At $\sim 1\,\mu\text{s}$ per $\mathbb{G}_1$ addition: $\approx 0.26\,\text{s}$.
- FFT: $O(C \log C)$ field operations, $\approx 0.1\,\text{s}$.
- **Total prover time: $\approx 0.4\,\text{s}$ on a single CPU core.**
- With GPU acceleration (e.g., `bellman` MSM on RTX 4090): ≈ 5–$15\,\text{ms}$.

Correction to earlier drafts. The $357\,\mu\text{s}$ figure cited in earlier drafts was unsupported by measurement. The closest real numbers are BLS single signing ($350\,\mu\text{s}$) and ML-DSA-65 signing ($495\,\mu\text{s}$). For the actual ML-DSA-65 verification circuit at $C \approx 2^{22.5}$, CPU prover time is $\sim 400\,\text{ms}$ and GPU prover time is ~ 5–$15\,\text{ms}$. Full QuasarCert CPU verification (BLS aggregated verify + Groth16 verify) is ~ 2–$5\,\text{ms}$. Z-Chain proof generation runs asynchronously (epoch-based, not per-block), so this does not impact finality latency. See `consensus/BENCHMARKS.md` for all measured values.

C Static vs Adaptive Corruption

Theorem 5 and its counterpart Theorem 7.5 in LP-020 assume *static* corruption: the adversary commits to the set of corrupted validators before protocol execution begins. This appendix addresses extension to the adaptive corruption model.

C.1 Corruption Models

Definition 13 (Static corruption [6])**.** *The adversary $\mathcal{A}$ selects the set $F \subset \mathcal{V}$ of corrupted parties before the protocol begins. $\mathcal{A}$ learns all internal state of parties in F but cannot corrupt additional parties during execution.*

Definition 14 (Adaptive corruption [6])**.** *The adversary $\mathcal{A}$ may corrupt parties at any point during protocol execution. Upon corrupting v_i, $\mathcal{A}$ learns all of v_i's current state (keys, randomness, signing transcripts). The total stake of corrupted parties remains bounded by τ.*

In the adaptive model, the simulator in a security proof must produce a consistent view for the adversary *after* the adversary chooses which party to corrupt. This is strictly harder than the static model: the simulator must either (a) program the random oracle to maintain consistency (programmable RO), or (b) ensure honest parties have erased sensitive state before corruption (secure erasures).

C.2 Static-Model Limitation

Remark 8. *Theorem 5 holds only under static corruption. The proofs of Corona SUF-CMA [2] and BLS EUF-CMA [1] both use rewinding-based extractors that require the corrupted set to be fixed before the simulation begins. In the adaptive model, the adversary can corrupt a validator* after *seeing its Corona Round 1 commitment, learning the signing randomness $\mathbf{y}_i$ and secret share $\mathbf{s}_i$.*

C.3 Extension via Programmable Random Oracle

Corollary 20 (Adaptive soundness, programmable RO)**.** *In the programmable random oracle model, Theorem 5 extends to adaptive corruption with a security loss of $O(n \cdot Q_H)$, where Q_H is the number of hash queries.*

Proof sketch. We apply the Fischlin transform [8]: the simulator programs the random oracle H so that upon adaptive corruption of v_i, the simulated signing transcripts remain consistent with the adversary's view. Per-component: BLS uses RO programming to embed a co-CDH challenge at the corruption point (loss $O(n)$); Corona programs H on the round-1 commitment so the adversary's view is extractable, then solves a linear system over R_q on corruption (loss $O(n)$); Groth16 in the AGM is rewinding-free and adaptively sound directly.

Combining with a union bound over n targets and Q_H queries:

$$\mathsf{Adv}^{\text{forge}}_{\text{adaptive}} \leq n \cdot Q_H \cdot \max(\epsilon_{\text{co-CDH}}, \epsilon_{\text{MLWE}}, \epsilon_{\text{MLWE/MSIS}})$$

which is negligible for $\lambda = 128$ when $n \cdot Q_H < 2^{60}$ (a generous bound for any practical deployment). □

C.4 Extension via Secure Erasures

Corollary 21 (Adaptive soundness, erasure model)**.** *If honest validators erase signing randomness ($\mathbf{y}_i$, PRF state) after each Corona round completes, then Theorem 5 extends to adaptive corruption without the programmable RO assumption, with no additional security loss.*

Proof sketch. If v_i erases $\mathbf{y}_i$ after broadcasting $\mathbf{z}_i$, then adaptive corruption of v_i reveals only $\mathbf{s}_i$ (the long-term share) and the completed signature transcript. The adversary gains no advantage beyond what a static adversary controlling v_i from the start would have. The static-model proof applies directly. □

Practical note. The Quasar implementation (Section 9 of LP-020) zeroes signing randomness after each round via Go's explicit memory clearing. This provides the erasure property in software. HSM-backed signing (Section 5 of LP-020, T-Chain) provides hardware-enforced erasure.

Blockchain corruption model. In blockchain settings, the standard corruption model is that of Garay–Kiayias–Leonardos [7]: the adversary is *mildly adaptive*, corrupting parties between rounds but not within a round. Both Cor. 20 and Cor. 21 cover this model as a special case.

D Trusted-Setup Ceremony

D.1 The SRS Assumption

Groth16 [4] requires a structured reference string (SRS) of the form:

$$\mathsf{SRS} = \left(\{g^{\tau^i}\}_{i=0}^{C},\ \{h^{\tau^i}\}_{i=0}^{C},\ \{g^{\alpha\tau^i}\},\ \{g^{\beta\tau^i}\},\ h^{\beta},\ h^{\gamma},\ h^{\delta}\right)$$

where $\tau, \alpha, \beta, \gamma, \delta$ are toxic waste that must be destroyed after generation. If any party learns τ, it can forge proofs for arbitrary false statements.

Circuit specificity. The Groth16 SRS is circuit-specific: a new SRS must be generated for each distinct R1CS circuit. For QuasarCert, the circuit is fixed (ML-DSA-65 verification for a given n), so a single ceremony per (n, ML-DSA parameters) tuple suffices.

D.2 Multi-Party Ceremony

The standard mitigation is an N-party ceremony where each participant P_j contributes randomness τ_j to the SRS. The final SRS uses $\tau = \sum_j \tau_j$ (or $\prod_j \tau_j$ in the multiplicative formulation). Security holds if *at least one* participant is honest and destroys their τ_j.

Theorem 22 (Ceremony security [9])**.** *An N-party Groth16 SRS ceremony is secure (no party learns τ) provided at least one participant honestly destroys their contribution. The ceremony can be run sequentially (each participant extends the previous SRS) with public verification of each step.*

For QuasarCert with $n = 21$ validators, the ceremony participants can be the validators themselves. With 21 participants, the probability that all are dishonest (and collude to reconstruct τ) is negligible under the $f < 1/3$ Byzantine assumption: at most 7 validators are corrupt, so at least 14 honestly destroy their contributions. The output is committed in zchain_vk_root on the Z-Chain genesis block.

D.3 Universal-Setup Alternatives

Universal-setup proof systems avoid circuit-specific ceremonies at the cost of larger proofs.

System	Proof size	Verify	Setup	Ref.
Groth16	192 B	3 pairings	Circuit-specific	[4]
PLONK	∼500 B	$O(1)$ pairings	Universal, updatable	[10]
Marlin	∼800 B	$O(\log C)$	Universal, updatable	[11]

Recommendation (3.0 final). Groth16 with a multi-party ceremony among the validator set, output committed in zchain_vk_root, is the production configuration for the 2025-12-25 activation. PLONK migration is tracked as a future-work item under LP-020 §11.

E Quantum-Safety Analysis

This appendix consolidates the concrete classical and quantum security estimates for each lane, then states the combined three-lane survival condition.

E.1 Lane 1: BLS12-381 (classical only)

Classical security is approximately 2^{128} (best classical attack on the discrete log in $\mathbb{F}_{q^{12}}$ via the special tower NFS). Quantum security is 0 in polynomial time once a cryptographically relevant quantum computer exists, by Shor on the underlying $\mathbb{G}_1, \mathbb{G}_2$.

E.2 Lane 2: Corona (Module-LWE / Module-SIS)

Parameter set from [2]:

Parameter	Symbol	Value
Ring dimension	N	256
Modulus	q	$2^{48} + 2561$ ($\approx 2^{48}$)
Module rows	M	8
Module columns	N'	7
Gaussian width (keygen)	σ_e	6.108
Gaussian width (signing)	σ^*	$2^{37.33}$
Rejection bound	B	$2^{48.61}$
Challenge weight	κ	23

The Module-LWE instance underlying Corona has effective lattice dimension $d = M \times N = 8 \times 256 = 2\,048$ over $\mathbb{Z}_q$.

Classical security via primal attack. Best classical attack: the primal lattice attack with BKZ-β sieving (BDGL [12]). Solving requires $\delta^{2\beta-d-1} q^{(d+1)/\beta} = \sigma_e \sqrt{2\pi e/(2\beta)}$ with root Hermite factor $\delta = \left((\pi\beta)^{1/\beta}\beta/(2\pi e)\right)^{1/(2(\beta-1))}$. For $d = 2048, q \approx 2^{48}, \sigma_e = 6.108$, numerics give $\beta \approx 430$. Classical cost

$$T_{\text{classical}} = 2^{0.292\beta + c_0} \approx 2^{0.292 \cdot 430 + 16.4} = 2^{142}.$$

Quantum security via Grover-sieving. The Laarhoven–BDGL exponent under Grover speedup is 0.265:

$$T_{\text{quantum}} = 2^{0.265 \cdot 430 + 16.4} = 2^{130.35}.$$

E.3 Lane 3a: ML-DSA-65 (FIPS 204)

Parameters: $k = 6$, $\ell = 5$, $q = 2^{23} - 2^{13} + 1$, $\eta = 4$. Effective MLWE dimension $5 \cdot 256 = 1280$, $\log_2 q = 23$. NIST Cat. 3 target: $\geq 2^{128}$ classical / $\geq 2^{128}$ quantum under the same NIST cost model [13]. Best known attack gives $\beta_{\text{BKZ}} \approx 360$, $T_{\text{quantum}} \geq 2^{128}$.

E.4 Lane 3b: Z-Chain Groth16

Classical: 2^{128} (BLS12-381 pairing-based, AGM/KEA). Quantum: Groth16 itself loses classical knowledge-soundness once Shor breaks discrete log on $\mathbb{G}_1$. However, the *statement proven* remains a conjunction of n ML-DSA-65 signature verifications, so a quantum adversary's effective task degrades to forging ML-DSA-65 signatures committed by `zchain_vk_root`. Standalone PQ security of Lane 3 thus tracks Lane 3a above.

A future-work direction is to swap Groth16 for a STARK-of-pairing or hash-based SNARK that is itself PQ-knowledge-sound. Tracked in LP-020 §11; not in 3.0 final scope.

E.5 Comparison table

Scheme	d	$\log_2 q$	β_{BKZ}	T_{quantum}
ML-DSA-65	1280	23	≈ 360	$\geq 2^{128}$
ML-KEM-768	768	12	≈ 340	$\geq 2^{128}$
Corona	2048	48	≈ 430	$\geq 2^{130}$
BLS12-381	–	–	–	0 (Shor)

E.6 Combined survival condition

The full QuasarCert quantum-safe predicate is $\mathsf{VerifyPQ} = \mathsf{RT.Verify} \wedge \mathsf{ZK.Verify}$ (Theorem 13). Equivalently, "1 of 2" PQ pillars must hold: either Corona's MLWE/MSIS, or the ML-DSA-65 path inside the Groth16 proof. The classical BLS lane provides classical-only defense in depth and contributes nothing to the PQ guarantee, in line with the analysis of pq-finality-no-bls.tex.

E.7 Caveats

1. BDGL sieving exponents 0.292 (classical) and 0.265 (quantum) are heuristic.
2. The Grover model assumes ideal oracle access; physical overhead increases effective security.
3. Ring structure of $\mathbb{Z}_q[X]/(X^{256}+1)$ has resisted known algebraic attacks at the relevant dimensions; an algorithmic breakthrough would require parameter revision.
4. Lane 3b PQ guarantee is conditional on the future-work SNARK swap. In 3.0 final, Lane 3 is treated as classical with PQ fallback to its statement.